
Rajshahi University of Engineering & Technology (RUET)

Department of Computer Science and Engineering (CSE)

CSE 2202: Algorithm Analysis and Design Lab

Lab Final Examination

Time: 1 Hour

Total Marks: 20

Problem: The RUET Rover's Mission-Critical Modules

Scenario:

The RUET Robotics Team is preparing for the National Autonomous Rover Competition. Their rover has a main processing unit with a very limited, high-speed cache of 10 MB.

The team has developed several data-processing "modules" (e.g., image recognition, sensor fusion, pathfinding). Each module provides a "Priority Score" (how critical it is for the mission), but also consumes a specific amount of the high-speed cache.

Your task is to write a program using **Dynamic Programming** to help the team select the optimal combination of modules that maximizes the total Priority Score without exceeding the 10 MB cache limit.

Given Data:

The available modules are:

Module	Cache Size (MB)	Priority Score
Module 1	6	60
Module 2	4	40
Module 3	3	20
Module 4	2	30

Your tasks are as follows:

1. (8 Marks) The DP Table Solution:

- Implement the bottom-up Dynamic Programming solution for the 0/1 problem.
- You **must** create a 2D DP table (e.g., $dp[n+1][W+1]$, where n is the number of modules and w is the total cache capacity).
- $dp[i][w]$ should store the maximum Priority Score achievable using a subset of the first i modules with a cache limit of w .
- **Output 1:** Print the **maximum possible Priority Score** that the rover can achieve (this will be the value in the last cell of your table, $dp[n][W]$).
- **Output 2:** For grading and verification, print the **entire completed 2D DP table**.

2. (6 Marks) Reconstructing the Solution:

- Implement the backtracking algorithm that traces through your completed DP table.
- This function must determine *which* modules were selected to achieve the maximum score found in Task 1.
- Start from $dp[n][W]$ and trace backward to find the set of included items.
- **Output 3:** Print the list of selected modules (e.g., "Module 1, Module 4").

3. (6 Marks) Analysis: DP vs. Greedy:

- **A) Greedy Calculation (3 marks):**
 - First, calculate the "Score-to-Size" ratio (Priority Score / Cache Size) for each module.

- Manually determine (or write a simple function) what the total score would be if you used a **greedy strategy**, always picking the module with the **highest ratio** first, until you run out of space.
 - **Output 4:** Print the total score achieved using this greedy strategy.
 - **B) Justification (3 marks):**
 - **Output 5:** Briefly (1-2 sentences) explain **why** the Dynamic Programming result (from Output 1) is different from the greedy result (from Output 4) and why the DP approach guarantees the true optimum, while the greedy one does not.
-

Implementation Guidelines:

- You must use a 2D array (or vector of vectors) for the DP table.
- Your code should be clean and well-commented.
- The solution reconstruction (Task 2) must algorithmically use the filled DP table from Task 1.